

Project Update Report: Autonomous Indoor Navigation Robot Using A* Search

Individual Progress Report — Md. Sadman Bin Murtoza (2232901642)

Project Overview

Our group is building a simulated robot that can find its way across an indoor floor plan and reach a target location while avoiding obstacles, using the A* search algorithm from this course. My individual responsibility is the path-planning core: the A* module that computes the shortest safe route from a start cell to a goal cell. This report covers what I personally worked on during the first two weeks of the project, from initial planning through a first working version of the search algorithm, alongside our group's full seven-week timeline for context.

Project Timeline (Weeks 1–7)

Week	Date	Planned / Completed Work
1	11 – 17 July	Define shortest-path requirements; set up support/astar.py as a separate module. (Completed — see details below)
2	18 – 24 July	Add grid states for start, goal, explored cells, and path; implement and test the full A* search. (Completed — see details below)
3	25 – 31 July	Refine and stress-test the completed A* module on trickier grids, ahead of the original schedule.
4	1 – 7 August	Hand the generated path to the robot movement system, with checks confirming it only contains valid grid cells.
5	8 – 14 August	Re-run A* from the robot's current position whenever a dynamic obstacle blocks the existing path.
6	15 – 21 August	Compute and supply path length, explored-node count, and calculation time to the interface.
7	22 – 28 August	Verify the algorithm against reachable, unreachable, short, and complex paths; remove unnecessary calculations.

Week 1 (11 – 17 July): Planning and Groundwork

The first part of this week was mostly about getting the project set up properly before writing any real code, so that the rest of the semester would go smoothly.

- Worked with my teammates to finalize the project scope: an indoor navigation robot that plans paths on a grid map using A* search.
- Did a short literature review comparing BFS, Dijkstra's algorithm, and A* for grid-based pathfinding, and noted why A*'s heuristic-guided search fits our use case better than the uninformed alternatives.
- Created our public GitHub repository and organized it to match the required structure: a main file, README.md, requirements.txt, and the data, support, and other sub-folders.
- Designed a simple 2D grid representation of a sample indoor floor plan (free cells, walls/obstacles, start cell, goal cell) and saved it as data/floorplan_v1.csv.
- Picked our initial tech stack — Python, NumPy, and Matplotlib — and drafted requirements.txt.
- Wrote the first version of README.md, covering the project idea, motivation, and repository layout.

Week 2 (18 – 24 July): First Working Implementation

Working ahead of schedule, I also completed the following Week 2 tasks alongside Week 1, so the group would have a working pathfinder in place early.

- Implemented the core A* search algorithm from scratch in support/astar.py: a priority queue of candidate cells ordered by cost-so-far plus a Manhattan-distance heuristic, expanding the lowest-cost cell first and skipping blocked or already-visited cells.

- Added internal tracking for the start cell, goal cell, explored cells, and the path found so far, and made sure the algorithm returns a clear “no path” result when the goal genuinely cannot be reached, not just when it succeeds.
- Wrote a small test script (support/test_astar.py) and checked the algorithm against three sample grids of increasing difficulty, confirming it always finds the correct shortest path.
- Built a visualization function (support/visualize.py) with Matplotlib that draws the grid, the obstacles, and the computed path, and saves it as an output image.
- Wired everything together in main.py, so running the program loads the sample floor plan and displays a complete, solved path end-to-end.
- Updated README.md with setup and usage instructions plus a sample output screenshot, and pushed all the work with clear, descriptive commit messages.
- Talked with my teammates about how the pathfinding module's output (the path and explored cells) will be consumed by the robot-movement and interface modules in the coming weeks.

Future Plans (Weeks 3–7)

With the core path-planning module already complete, the remaining weeks will shift from building it to refining it, integrating it with my teammates' modules, and testing the full system.

- Week 3 (25–31 July): Refine and stress-test the completed A* module on trickier grids — narrow corridors, larger maps, and cases with multiple equally short paths — before other modules start depending on it.
- Week 4 (1–7 August): Hand the generated path over to the robot-movement module my teammate is building, and add a validation step confirming every cell in a returned path is free and inside the grid boundaries.
- Week 5 (8–14 August): Make the algorithm re-plan on demand — re-running A* from the robot's current position whenever a dynamic obstacle blocks the existing route — and hand back a fresh route around it.
- Week 6 (15–21 August): Add simple performance reporting (path length, explored-node count, and calculation time) and expose these numbers to the interface my teammate is building, for our demo.
- Week 7 (22–28 August): Carry out final stress-testing across small and large grids, reachable and unreachable goals, and short and complex paths; clean up the code and comments; and prepare documentation for submission.

Expected Outcome

By the end of the project, my A* module will reliably compute the shortest safe path between any two reachable cells on our indoor grid, correctly report when no path exists, and re-plan on the fly when a new obstacle appears mid-route. It will expose path length, explored-node count, and timing information for the rest of the team's interface, and will have been tested against maps of increasing difficulty. Working on this module is deepening my understanding of heuristic search, priority-queue-based algorithms, and how to design a component the rest of the team's code can plug into cleanly.

GitHub Repository

All code, data, and documentation described above have been pushed to our group's public GitHub repository, organized according to the course's required folder structure, with commit history reflecting steady, ongoing weekly progress.

Conclusion

Two weeks into the semester, the path-planning core of our indoor-navigation robot is already implemented, tested, and visualized, ahead of the group's original schedule. This gives the team a working A* module to build on early, so the remaining weeks can focus on integrating it with the robot-movement and interface components my teammates are developing, adding dynamic re-planning, and thoroughly testing the complete system before final submission.